PRODUCT ASSIGNMENT[TREES'S 31/08/26]

UDAY CHIRRA
2303A510G2

1. Boundary Traversal

```
class Main {
   static class Node {
      int data;
      Node left, right;

      Node(int data) {
         this.data = data;
      }
   }

   static void printLeft(Node root) {
      if (root == null)
         return;

      if (root.left != null || root.right != null) {
         System.out.print(root.data + " ");

         if (root.left != null)
            printLeft(root.left);
         else
            printLeft(root.right);
      }
   }

   static void printLeaves(Node root) {
      if (root == null)
         return;

      if (root.left == null && root.right == null) {
         System.out.print(root.data + " ");
         return;
      }
```

```
        printLeaves(root.left);
        printLeaves(root.right);
    }

    static void printRight(Node root) {
        if (root == null)
            return;

        if (root.left != null || root.right != null) {
            if (root.right != null)
                printRight(root.right);
            else
                printRight(root.left);

            System.out.print(root.data + " ");
        }
    }

    static void boundary(Node root) {
        if (root == null)
            return;
        System.out.print(root.data + " ");
        printLeft(root.left);
        printLeaves(root.left);
        printLeaves(root.right);
        printRight(root.right);
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);
        root.right.left = new Node(6);
        root.right.right = new Node(7);

        boundary(root);
    }
}
```

2. Left View

```
import java.util.*;

class Main {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }

    static void leftView(Node root) {
        if (root == null)
            return;

        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            int n = queue.size();

            for (int i = 0; i < n; i++) {
                Node current = queue.poll();

                if (i == 0)
                    System.out.print(current.data + " ");

                if (current.left != null)
                    queue.add(current.left);

                if (current.right != null)
```

```
                queue.add(current.right);
            }
        }
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);
        root.right.right = new Node(6);

        leftView(root);
    }
}
```

3. Right View

```
import java.util.*;

class Main {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }

    static void rightView(Node root) {
        if (root == null)
            return;

        Queue<Node> queue = new LinkedList<>();
        queue.add(root);
```

```
        while (!queue.isEmpty()) {
            int n = queue.size();

            for (int i = 0; i < n; i++) {
                Node current = queue.poll();

                if (i == n - 1)
                    System.out.print(current.data + " ");

                if (current.left != null)
                    queue.add(current.left);

                if (current.right != null)
                    queue.add(current.right);
            }
        }
    }

    public static void main(String[] args) {
        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);
        root.right.right = new Node(6);

        rightView(root);
    }
}
```

6. Construct Tree from Inorder and Preorder

```
class Main {
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
```

```
    }

    static int index = 0;

    static Node buildTree(int[] preorder, int[] inorder, int start, int end) {
        if (start > end)
            return null;

        Node root = new Node(preorder[index++]);

        int position = start;

        while (inorder[position] != root.data)
            position++;

        root.left = buildTree(preorder, inorder, start, position - 1);
        root.right = buildTree(preorder, inorder, position + 1, end);

        return root;
    }

    static void postorder(Node root) {
        if (root == null)
            return;
        postorder(root.left);
        postorder(root.right);
        System.out.print(root.data + " ");
    }
    public static void main(String[] args) {
        int[] preorder = {1, 2, 4, 5, 3, 6, 7};
        int[] inorder = {4, 2, 5, 1, 6, 3, 7};

        Node root = buildTree(preorder, inorder, 0, inorder.length - 1);

        postorder(root);
    }
}
```